

Assignment 07: Data classes

Introduction

In module 07, we learned about class inheritance, overriding default or inheritance methods, using GitHub Desktop, constructors, attributes, and properties.

What we're doing in this assignment: *Create a Python program that demonstrates using constants, variables, and print statements to display a message about a student's registration for a Python course.*

What changed from last assignment: *This program is very similar to Assignment05, but it **adds set of data classes**.*

What we will discuss in this document is the `__init__` method, the keyword "self", getters and setters, the `__str__` method, and class inheritance.

`__init__` constructor

In the Person and Student classes, there is a special method called `__init__`. Below is the `__init__` method that is in the Person class.

```
def __init__(self, first_name: str = '', last_name: str = ''):
    self.first_name = first_name
    self.last_name = last_name
```

`__init__` is a constructor and 'init' is short for 'initializer'. "It is used to initialize the attributes of the object with the values provided at the time of object creation" (*Python Classes and Objects*, Geeks for Geeks, external link: <https://www.geeksforgeeks.org/python/python-classes-and-objects/>). When an object is created, the `__init__` method is executed automatically. It constructs the object by inserting the values in the appropriate attributes.

"self"

Throughout the Person and Student classes, the keyword "self" is used frequently. Below is an excerpt of the Student class, emphasizing instances of "self."

```
class Student(Person):

    def __init__(self, first_name: str = '', last_name: str = '',
                  course_name: str = ''):
        super().__init__(first_name=first_name, last_name=last_name)
        self.course_name = course_name

    @property
    def course_name(self):
        return self.__course_name

    @course_name.setter
    def course_name(self, value: str):
        self.__course_name = value

    def __str__(self):
        return f'{self.first_name},{self.last_name},{self.__course_name}'
```

The only lines in the code above that don't use the keyword "self" are the class name, decorators or the call to the Superclass Person to initialize first_name and last_name.

In other programming languages, "self" is sometimes called "this" (Mod07-Notes.pdf, page 9). Thinking of "self" through the lens of "this" has helped me grasp this concept better. Python wants to know specifically which object we are referring to when handling methods in a data class. In order to specify which object, we use "self" to say ***"this object"***.

For example, the line of code: "self.course_name = course_name" would mean... give ***this object*** (or ***this Student***... since we are dealing with the Student object class) their own course name.

The next line: "return self.__course_name" would mean... return the course name that belongs to ***this object/this student***.

Properties: Getter and setters

In the Person class, below the __init__ method, there are getters and setters for both the first_name property and last_name property (shown below).

```
@property # Getter/accessor
def first_name(self):
    return self.__first_name.title() # Formatting code

@first_name.setter # Setter
def first_name(self, value:str):
    if value.isalpha() or value == "": # is character or empty string
```

```

        self.__first_name = value
    else:
        raise ValueError("The first name is restricted to alphabetic characters
only.")

@property # Getter/accessor
def last_name(self):
    return self.__last_name.title() # Formatting code

@last_name.setter # Setter
def last_name(self, value: str):
    if value.isalpha() or value == "": # Is character or empty string
        self.__last_name = value
    else:
        raise ValueError("The last name is restricted to alphabetic characters
only.")

```

First, what are properties? “Properties are functions designed to manage attribute data” (Mod07-Notes.pdf, page 15).

“Typically, for each attribute, you create two properties—one for ‘getting’ data and one for ‘setting’ data” (Mod07-Notes.pdf, page 15). This is where the “Getters” and “Setters” come in. The formal names for these are “accessors” and “mutators” and carry decorators of @property and @name_of_property.setter, respectively (Mod07-Notes.pdf, page 15). Broken down, they are special methods that help you to safely *get/access a value* (Getter/Accessor) and *set a value* (Setter/Mutator).

__str__ method

What is the __str__ method? “The __str__ method in Python is responsible for returning a human-readable string representation of an object” (Mod07-Notes.pdf, page 22). In essence, it is meant to return a user-friendly description of the object.

We have already seen calls to this method in previous module’s labs and notes when involving code related to printing custom error messages. But in this module, we learned what it is and how to utilize creating custom __str__ methods in both the Person and Student classes.

If we do not include a __str__ method in a class, an ‘invisible’ __str__ method still exists. “[B]y default, the __str__() method in your class, Python’s default implementation of __str__() will return a string that includes the object’s memory address” (Mod07-Notes.pdf, page 22). But we have the capacity to customize what is printed when we call the object of that class.

The below code is the __str__ method in our Person class.

```
def __str__(self):  
    return f'{self.first_name},{self.last_name}'
```

By customizing the `__str__` method in this code, we have changed what is printed when we call the `Person` object. Instead printing the object location (the default `__str__`), it now will print the `Person`'s first name and last name (in a comma-separated format). "By defining a custom `__str__()` method, you can control what is displayed when you print an object of your class, making it more user-friendly and meaningful" (Mod07-Notes.pdf, page 22).

In the `Student` class, we have to override the inherited `__str__` method but we do so the same way we override the default `__str__` method.

```
def __str__(self):  
    return f'{self.first_name},{self.last_name},{self.__course_name}'
```

This code customizes the `__str__` method to display a message of the `Student`'s first name, last name, and course (in a comma-separated format).

Class Inheritance

Class inheritance is an important concept where "a new class (child class) can inherit data and behaviors (properties and methods) from an existing class (parent class)" (Mod07-Notes.pdf, page 21). In the `Student` class, we implement this concept of by making it a subclass of the `Person` class.

```
class Student(Person):  
  
    def __init__(self, first_name: str = '', last_name: str = '',  
                  course_name: str = ''):  
        # self.first_name = first_name  
        # self.last_name = last_name  
        super().__init__(first_name=first_name, last_name=last_name)  
        self.course_name = course_name  
  
    @property  
    def course_name(self):  
        return self.__course_name  
  
    @course_name.setter  
    def course_name(self, value: str):  
        self.__course_name = value
```

```
def __str__(self):  
    return f'{self.first_name},{self.last_name},{self.__course_name}'
```

Within the Student class, we make calls to the Superclass (Person) for data and behaviors related to first name and last name.

Summary

In this assignment, we added two new data classes: Person and Student. In each class, we wrote the `__init__` method, used the keyword “self”, added getters and setters, and used the `__str__` method. We learned about inheritance through the Student class inheriting `first_name` and `last_name` code from the Person class.